

Mobility-Aware Multi-hop Task Offloading for Autonomous Driving in Vehicular Edge Computing and Networks

Lei Liu, Ming Zhao, Miao Yu, Mian Ahmad Jan, Dapeng Lan and Amirhosein Taherkordi

Abstract—Vehicular Edge Computing (VEC) has gained significant interest due to its potential to provide low latency and reduce the load in backhaul networks. To meet the increasing computational demands of emerging and ever-growing vehicular applications, e.g. autonomous driving, abundance in computation resources of individual vehicles can play a crucial role in task execution within a VEC scenario, that can further contribute towards improving the user experience. This is however an extremely challenging task due to high mobility of vehicles that can easily lead to intermittent connectivity, thereby disrupting on-going task processing. In this paper, we propose a task offloading scheme by exploiting multi-hop vehicle computation resources in VEC based on mobility analysis of vehicles. In addition to the vehicles within one hop from the task vehicle that generates computation tasks, certain multi-hop vehicles that meet the given requirements in terms of link connectivity and computation capacity, are also leveraged to carry out the tasks offloaded by the task vehicle. An optimization problem is formulated for the task vehicle to minimize the weighted sum of execution time and computation cost of all tasks. A semidefinite relaxation approach with an adaptive adjustment procedure is proposed to solve the formulated optimization problem for obtaining the corresponding offloading decisions. The simulation results show that our proposed offloading scheme can achieve significant improvement in terms of response delay by at least 34% compared with the existing algorithms, i.e., local processing and random offloading.

Index Terms—Vehicular edge computing, task offloading, multi-hop, mobility, connectivity.

I. INTRODUCTION

With the advancement of information and communication technologies [1]–[4], Internet of Vehicles (IoV) has attracted considerable attention in recent years from the research community [5], [6], and has witnessed a remarkable development along with the availability of more advanced devices and technologies [7], [8]. This has led to crucial developments in a variety of vehicular applications aimed at improving road safety, enhancing traffic efficiency and providing convenient infotainment services to the users [9]. Nonetheless, such

applications, particularly those with strict latency constraints and intensive processing, such as autonomous driving, impose considerable demands on computation and storage resources. [10]. To meet these demands, Vehicular Edge Computing (VEC) is viewed as a promising approach [11], [12]. With the integration of Mobile Edge Computing (MEC) [13]–[15] and vehicular networks, VEC aims at reducing the response time and alleviating backhaul network load by moving the computation and storage resources in proximity to vehicular users [16]. However, due to unique characteristics of vehicular networks [17], especially the high mobility of nodes and varying channel conditions, designing an efficient edge-enabled task offloading scheme is quite challenging.

There have been numerous efforts devoted to task offloading in VEC, most of which focus on investigating server-based offloading strategy where Road Side Units (RSUs) with certain communication, computation and storage resources mainly act as edge servers. In [18], workload scheduling was implemented on a single edge server. The work in [19] considered a multi-server network scenario. In [20] and [21], mobility-based task offloading among multiple edge servers was studied. However, the penetration of edge servers is low at initial stages of deployment. Besides, the deployment of edge servers can be quite expensive. From a business perspective, it is impractical to fully deploy edge servers along the road segments, especially in a large-scale network. Edge servers, on the other hand, struggle to keep up with rapidly growing traffic data due to restricted computation capability when compared to the cloud.

Vehicle-to-Vehicle (V2V) offloading is an exciting solution for dealing with the limitations of server-based offloading and mitigating the scarcity of processing resources in VEC. The increasing growth in the processing speed and storage capacity allow the smart and connected vehicles to communicate as well as permit them to have some computation and cache resources for task processing. The presence of large underutilized resources in individual vehicles presents a huge opportunity for improving vehicular networks' communication and processing capability. The aggregation of these abundant idle resources can improve resource utilization and user experience. All these factors provide a powerful platform for exploiting the vehicle resources. V2V offloading, in particular, benefits from being close to vehicular users, allowing proximity services to be provided through resource sharing.

In literature, there has been few studies conducted for designing an efficient V2V offloading schemes. Some of these

Lei Liu and Ming Zhao are with the State Key Laboratory of Integrated Service Networks, Xidian University, Xi'an 710071, China (e-mail: tianjiaoliulei@163.com; zm227926019@163.com.)

Miao Yu is with the College of Textiles and Clothing, Qingdao University, Qingdao 266071, China (e-mail: yumiao@qdu.edu.cn.)

Mian Ahmad Jan is with the Department of Computer Science, Abdul Wali Khan University Mardan, Pakistan (e-mail: mianjan@awkum.edu.pk.)

Dapeng Lan and Amirhosein Taherkordi are with the Department of Informatics, University of Oslo, Norway (email: dapengl@ifi.uio.no; amirhost@ifi.uio.no).

Corresponding Author: Mian Ahmad Jan

studies make use of the computation resources of vehicles for service provisioning, but fail to fully capture the mobility of vehicles and are based on the assumption that all the vehicles are willing to share resources for free, e.g., [22]. Specifically, most of existing work on V2V offloading mainly depend on one-hop vehicles for task execution, ignoring the utilization of available multi-hop vehicles. Based on this statement, there is an urgent need to design an efficient V2V offloading scheme. To this end, the following factors should be well considered for any such design:

- 1) Resource discovery. The mobility and computational capacity for each vehicle are different. Multi-hop link, on the other hand, is vulnerable because of the complicated channel environment and dynamic changes in topology. Therefore, it is crucial to find available vehicles for task processing.
- 2) Response latency. Due to the computational and transmission needs, response delay is regarded as the most demanding metric to measure the quality of service. Thus, minimizing the response delay is one of the main targets for an offloading scheme.
- 3) Incentive scheme. Considering the selfishness of nodes, the owners of computation and storage resources may not be willing to rent out their resources unless they get paid well. An incentive strategy is therefore necessary to encourage resource sharing among the nodes.
- 4) Mobility. Suffering from the high mobility of nodes, the network topology varies with time, which introduces difficulties in providing optimal decision making. In addition, due to the short contact time between nodes, a successful offloading process should guarantee that the task offloaded from a task vehicle to its service vehicle should be completed before the link between them becomes broken.

To address these issues, we propose a Mobility-Aware Multi-hop Task Offloading (MMTO) in vehicular edge computing, with the aim to maximize resource utilization and enhance user experience by making the best use of the underutilized computation resources of vehicles. Its novelty is that the offloading policy is constructed by addressing the difficulties in task assignment caused by mobility for one-hop service vehicles and taking advantage of multi-hop service vehicles for task computation. For one-hop service vehicles, the assigned tasks need to be finished before they move out of the transmission range of the task vehicle. As for multi-hop service vehicles, they are capable of participating in task processing only if the link connectivity between the task vehicle and the service vehicle is available before the calculation results are returned to the task vehicle. Based on the proposed task offloading scheme, a mixed-integer program is formulated. To provide a solution to this problem, a semidefinite relaxation approach is proposed along with an adaptive adjustment procedure considering the connectivity constraints on one-hop and multi-hop service vehicles. The contributions of our work are summarized as follows.

- A task offloading scheme is proposed by utilizing multi-hop vehicles which meet the given requirements regard-

ing mobility pattern and computation capacity, in addition to one hop vehicles of the task vehicle.

- An offloading policy is designed considering the response delay and economic cost to minimize the utility of task vehicle, subject to the connectivity constraints.
- A semidefinite relaxation (SDR) approach with an adaptive adjustment scheme is presented to solve the formulated optimization problem by considering the connectivity constraints of one-hop and multi-hop service vehicles while offloading the tasks.

The remainder of this paper is organized as follows. Section II gives an introduction of some existing work in VEC. Section III presents the system overview of our work. The exploiting mobility task offloading algorithm is proposed in Section IV. Numerical results are given in Section V, followed by conclusion in Section VI.

II. RELATED WORK

In literature, a number of studies have been devoted to the research on vehicle edge computing (VEC), some solid works among which are introduced in this section.

In VEC, edge servers with certain computation and cache resources, e.g., RSUs, are mainly used for vehicular users to execute certain tasks. The work in [18] investigated workload offloading and task scheduling by accounting for the high mobility of vehicles and their sojourn time within the communication range of edge servers. In [23], a dual-side optimization problem was formulated with the aim to balance the interests of both vehicles and edge servers at the same time. The computation offloading behavior of different vehicles is regarded as a competition for an edge server's resources in [24], where vehicles are to allowed to adjust their offloading probabilities in the pursuit of maximizing the utility. The works in [18], [23], [24] mainly focused on a single edge server. On the other hand, the works of [19], [25]–[27] focused on multi-server scenarios. An architecture for vehicle-infrastructure communication with high reliability and low latency was proposed in [19], which optimizes the association of vehicle and a small base station along with wireless resource management. In [25], the designed task offloading scheme simultaneously optimizes the selection of target server and the determination of transmission model. Communication, caching and computation are jointly taken into account to reduce the delay in [26]. Similar to [26], the authors utilized a deep reinforcement learning approach to solve the resource allocation problem with the consideration of networking, caching and computation in [27]. The work of [28] aimed at the joint optimization of task offloading and resource management while taking into account the differential privacy as well.

The popularization and intelligence of vehicles make it possible to fully exploit idle vehicle computation and cache resources for service provisioning. Owing to the remarkable characteristics including large number of vehicles, long sojourn time as well as specific staying locations, parked vehicles are regarded as a good choice to serve as service infrastructures. Clusters of parked vehicles were used to

act as virtual RSUs in [29], which is of significant benefit in coping with severe network fragmentation in vehicular networks. The work of [29] studied two key questions: the selection of gateway nodes in virtual clusters and seamless handovers among these gateways. Content downloading is investigated in [30], where parked vehicles are utilized to extend the service coverage of RSUs. Optimization models are presented to maximize the content freshness, the resource utilization as well as the fairness of exploiting the energy resources. The authors in [31] proposed a cooperative and distributed computation framework, where vehicles with some computation resources act as edge nodes. A vehicle-assisted offloading algorithm was presented in [22], where vehicles are exploited for providing computation services. With the aim to minimize the average offloading delay, an adaptive learning-based task offloading scheme was proposed in [32]. The delay performance of one-hop service vehicles is learnt by the task vehicle in a distributed manner. A V2V task offloading framework was presented in [33], where the computation resources of available surrounding vehicles can be utilized by the task vehicle. Specially, context-aware opportunistic offloading algorithms are implemented with the consideration of vehicle speed, direction and location to deal with mobility issues. In [34], a k-hop-limited V2I vehicular data offloading scheme was designed. The novelty of this scheme lies in finding the multi-hop V2V path by which the front or the rear RSU can be connected from the task vehicle, thereby increasing the possibility of vehicular data offloading. The novelty of the designed task offloading strategy in [35] lies in combining the dual features of V2V, i.e., communication and sensing. The scheduling policy, bandwidth ratio and transmission choice are jointly optimized for the purpose of minimizing the total response latency.

In VEC, the high mobility of vehicles should be well addressed, which will impact the success of task offloading. When a task is offloaded by a task vehicle to an edge server, it is likely that the task vehicle needs to pass through multiple edge servers before the task is finished. In order to deal with this issue, a mobility-based task offloading is agreed as a promising approach [20] [21]. A prediction-based transmission scheme was proposed in [20] for task processing with the consideration of mobility of vehicles, which enables the improvement of transmission efficiency while meeting the delay requirement. The work in [21] studies the problem of resource allocation in a multi-user multi-server network scenario by integrating load balancing with task offloading. Each vehicle can offload its task to a desired server for computation through movement. In [36], a generic relative motion model was provided to design a link stability metric by which the most stable vehicles surrounding the task vehicle are discovered. Among these stable vehicles, the vehicle which meets the task vehicle's preferences and the service constraints is selected as the optimal service provider. The authors in [37] designed a mobility-oriented retrieval scheme, where vehicles and RSUs are used for the communication nodes. In this scheme, the processed data can be effectively retrieved. An efficient mobility-based task offloading scheme was investigated in [38], which aims at the selection of offloading time

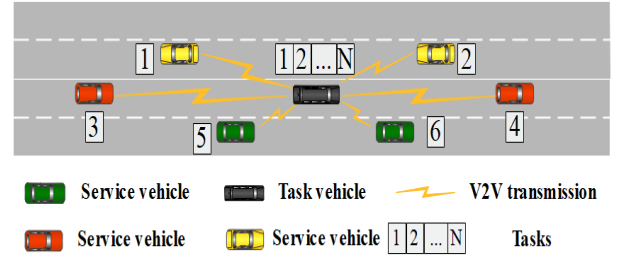


Fig. 1: Network scenario

and the allocation of communication, computing and caching resources with the consideration of the vehicle mobility and the latency constraint. In [39], the random mobility of vehicles and the potential handover during the offloading process are considered while designing a mobility-aware offloading algorithm.

A comparison of the above mentioned works is presented in Table I. In summary, most of the existing works on task offloading in VEC are assisted by edge servers. While mobility has been viewed as a challenge, we introduce a new perspective that enables us to exploit the link connectivity for optimizing task offloading further. In particular, we include multi-hop vehicles of the task vehicle in addition to one-hop vehicles for task offloading. In this regard, our work is one of the first to explore underutilized computation resources in individual vehicles to enhance the edge capacity by considering the potential opportunity of multi-hop vehicles on task implementation, which is beneficial for guaranteeing the quality of service of vehicular users.

III. SYSTEM OVERVIEW

Our work focuses on V2V offloading as illustrated in Fig.1. The considered scenario is a straight road, e.g., a highway, where vehicles are moving in two different directions. The main symbols used in this paper are summarized in Table II.

In the proposed scenario, there are two types of vehicles, i.e., client vehicles and service vehicles. They are classified based on their roles in task processing. Vehicles which generate tasks and need to process these tasks are defined as client vehicles, while vehicles which have surplus idle computation resources and are willing to share their resources are defined as service vehicles. Note that the role of each vehicle can change with time, which is dependent on whether it has a task to be executed or has abundant computation resources to share.

A vehicular application may consist of multiple services. For example, AR-based driving assistance application is composed of object recognition and video streaming as introduced in [40]. Each service has its attributes, including data size, computation complexity, etc. Here, we regard each service as a task. For a client vehicle, its tasks can be processed by the following two strategies: (a) Computed locally: the client vehicle processes its tasks by itself. (b) Offloaded to nearby vehicles: the client vehicle uploads its tasks to its neighboring vehicles with surplus computation resources for task execution.

TABLE I: COMPARISON of EXISTING RELATED WORK

Reference	Offloading Model	Key Metric	Main Contribution
[18]	Vehicle-to-RSU	The rejection rate of requests	Investigate the workload offloading and scheduling using edge cloud capacities
[23]	Vehicle-to-RSU	Cost and Profit	Minimize the cost of vehicles and edge servers simultaneously by formulating a dual-side optimization problem
[24]	Vehicle-to-RSU	Offloading probability, payoff and latency	Study the distributed computation offloading method by using a multi-user noncooperative game
[25]	Vehicle-to-RSU	Offloading utility	Leverage the deep learning approach to jointly optimize the selection of edge server and the determination of data transmission strategy
[26]	Vehicle-to-RSU	Delay and cache hit ratio	Provide a joint communication, caching and computation approach to reduce the delay for infotainment content retrieval
[27]	Vehicle-to-RSU	Total utility	Design an integrated architecture for dynamically managing networking, cache and computation resources
blue [28]	Vehicle-to-RSU	Average delay reduction	Make the joint optimization of task offloading and resource management with the consideration of differential privacy
[29]	Vehicle-to-Vehicle	Success ratio, MAC busy-fraction and number of handovers	Make clusters of vehicles to serve as virtual server unit
[30]	Vehicle-to-Vehicle	freshness	Investigate the content downloading with the assistance of parked cars which extend the service convergence of edge servers
[31]	Vehicle-to-Vehicle	Formulation latency, performance gain, perception reaction time	Provide a collaborative task offloading scheme for task processing
[22]	Vehicle-to-Vehicle	Utility function and processing Latency	Explore vehicle resources to design an offloading scheme with the consideration of processing latency
[32]	Vehicle-to-Vehicle	Delay	Develop an adaptive learning task offloading scheme by using multi-armed bandit algorithm
[33]	Vehicle-to-Vehicle	Efficiency, average waiting time, failure rate	Devise a context-aware opportunistic task offloading scheme which takes full use of available vehicle resources
[34]	Vehicle-to-Vehicle	Average path lifetime and average session time	Integrate vehicle-to-infrastructure offloading and vehicle-to-vehicle offloading for task processing
blue [35]	Vehicle-to-Vehicle	Overall response time	Jointly optimize the scheduling policy, bandwidth ratio and transmission choice for the purpose of minimizing the total response latency
[20]	Mobility-aware	Offloading time cost	Present a prediction based offloading scheme which can address the offloading time cost and the mobility
[21]	Mobility-aware	Cost and Profit	Integrate load balancing and task offloading for resource management
[36]	Mobility-aware	Average link stability derivation, provider vehicle duration, service delay, successful execution ratio	Investigates the service selection by link stability metric and fuzzy quantification
[37]	Mobility-aware	Delay	Provide a data retrieval strategy based on mobility for computation offloading
[38]	Mobility-aware	System cost	Investigate the task offloading strategy with the aim to make the minimization of the system cost subjecting to the maximum latency and other constraints
[39]	Mobility-aware	Cost and cost reduction	Develop the mobility-aware computation offloading with the consideration of random mobility and potential handover of vehicles when the offloading is being implemented
Our work	Mobility-aware	Delay and cost	Provide a mobility-aware multi-hop task offloading to improve the quality of service by exploring the computation resources of one-hop and multi-hop vehicles

TABLE II: Main Symbols Used in This Paper

Notation	Description
R	Transmission range
r_{ij}	The data transmission ratio between vehicle i and vehicle j
l_{ij}	The connection time between vehicle i and vehicle j
o	The client vehicle which have some tasks to implement
$\mathcal{N}$	The set of all the tasks from the client vehicle
$\mathcal{M}$	The set of all the available service vehicles
s_i	The input data size from vehicle i
c_i	The required computational resource for processing the task from vehicle i
x_{0i}	Whether task i is processed locally
x_{ki}	Whether task i is processed by service vehicle k
T_{0i}	The time required for client vehicle o to execute task i locally
T_{ki}	The time required for service vehicle k to execute task i
p_{ok}	The connectivity of the optimal offloading path between client vehicle o and service vehicle k
C_{ki}	The total cost needed for service vehicle k to process task i
$\mathbf{X}$	The task allocation matrix
$t(\mathbf{X})$	The time for processing all the tasks from client vehicle
$c(\mathbf{X})$	The cost for processing all the tasks from client vehicle
$\varphi(\mathbf{X})$	The offloading utility of client vehicle
λ_c	The weight factor

The general process of task processing includes: discover service vehicles, allocate client vehicle's tasks and implement computation tasks, which are described in detail as below.

1) *Service Vehicle Discovery*: In vehicular networks, vehicles exchange the speed and position information with their neighbors by periodically broadcasting beacons. For service vehicles, they can announce their availability by adding the computation information in the sent beacons. When a client vehicle needs to process the generated tasks, it listens to the beacons from its neighboring vehicles to find some available service vehicles for task offloading.

(a) One-hop service vehicles: For a client vehicle, vehicles within its one-hop range are called as one-hop vehicles, which can act as candidate service vehicle. This is because the client vehicle can directly communicate with each one-hop vehicle, which enables the task offloading between them. As shown in Fig. 2, vehicle 2 and vehicle 3 are within the communication range of client vehicle 1. In this case, vehicle 1 can directly upload its tasks to vehicle 2 and vehicle 3 for processing if needed.



Fig. 2: Example of service vehicles

(b) Multi-hop service vehicles: Vehicles within the multi-hop range of the client vehicle are also potential service vehicles. This is based on the fact that these multi-hop vehicles can connect to the client vehicle with the multi-

hop transmission of intermediate vehicles. At the initial stage, the client vehicle cannot directly communicate with its multi-hop vehicles due to the limitation of inter-vehicle distance. However, it can obtain the information (e.g., speed, position and computation resources) of its multi-hop vehicles assisted by its intermediate vehicles which can serve as relay nodes. The client vehicle broadcasts the service requirements for task execution (e.g., delay and energy consumption). After receiving this information, each one-hop vehicle will continue to transmit it onward when the link connectivity satisfies certain conditions. On the other hand, the one-hop vehicle can reply the client vehicle for being service vehicle along the reverse path if it fulfills the service requirements. This procedure is repeated until reaching the maximal hop number. Given a service vehicle, more than one routing paths may exist from itself to the client vehicle. The routing path with best transmission performance can be determined as the optimal one for data transmission. Based on the collected information related to multi-hop vehicles, the client vehicle is able to judge if these multi-hop vehicles are available. If a multi-hop vehicle is selected as the service vehicle to help the client vehicle in task processing, it first receives the offloaded tasks from the client vehicle with the assistance of intermediate relay nodes and then participates in task processing. For example, vehicle 1 broadcasts the offloaded tasks in Fig. 2. Upon the reception of these tasks, one-hop vehicle 2 and one-hop vehicle 3 are responsible for relaying these tasks to two-hop vehicles 4 and 5, respectively. The selected two-hop service vehicle executes the assigned tasks and then transmits the calculated results to the client vehicle using a multi-hop transmission.

2) *Task Allocation of Client Vehicles*: Upon collecting the information of available service vehicles, the client vehicle needs to design an efficient task allocation algorithm to decide where and how to run its tasks to meet its requirements.

(a) The design of optimization problem: In order to make the optimal offloading decisions, an optimization problem is formulated where the processing delay and the incentive pay are considered. The delay includes the uploading delay by wireless transmission, computing delay at vehicles, and downloading delay through wireless transmission. Different from traditional MEC, the total delay in VEC is also affected by the inter-vehicle distance between the client vehicle and its service vehicles. With the aim to improve the utilization of vehicular resources and the experience of vehicular users, an incentive scheme is necessary to encourage more vehicles with abundant idle computation resources to participate in task offloading. For the client vehicle, it needs to take into account the payment when resorting to service vehicles for task processing while satisfying the applicable requirements.

(b) Constraints: The link connectivity plays a crucial role in guaranteeing task offloading. Due to the mobility of nodes, frequent link disruptions are unavoidable in vehicular networks. Thus, the link connectivity between the client vehicle and its one-hop service vehicles will considerably impact the task allocation. When the client vehicle offloads the tasks to its one-hop service vehicles for task processing, the better the link connectivity, the larger the successful offloading probability will be. As shown in Fig. 3, a client vehicle A uploads some

tasks to its one-hop vehicle C. However, as the short contact duration suffers from mobility, there is a possibility that when vehicle C completes executing the offloaded tasks from vehicle A, it has moved out of the transmission range of client vehicle A. In this case, the calculation result cannot be sent to client vehicle A. In order to guarantee the successful offloading process, the offloaded tasks should be finished before the link between the client vehicle and its one-hop service vehicle becomes disconnected.

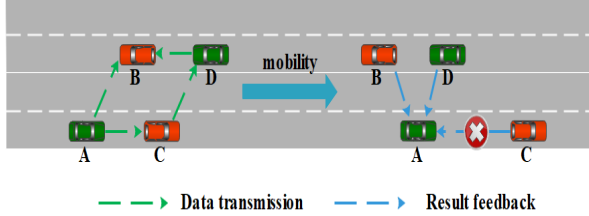


Fig. 3: Example of vehicle mobility

The link connectivity should also be considered for multi-hop service vehicles. Different from the link between the client and its one-hop service vehicle, the one between the client and its multi-hop service vehicle is more likely to be interrupted. An available multi-hop service vehicle can obtain the assigned tasks from the client vehicle through multi-hop wireless transmission, and has the chance to send the result to the client vehicle if the link between the client vehicle and the multi-hop service vehicle remains connected during the offloading process.

3) *The Implementation of Computation Tasks:* By solving the formulated problem, the client vehicle can obtain the optimal offloading decisions. Based on the offloading decisions, if it is necessary to depend on some service vehicles for task processing, the client vehicle transmits its tasks to its service vehicles. Once these task are received, each service vehicle begins to participate in task processing. After finishing the assigned tasks, service vehicles send the results back. Receiving all the results, the client vehicle can successfully run its application. The entire flow chart of the proposed offloading scheme above is illustrated in Fig.4.

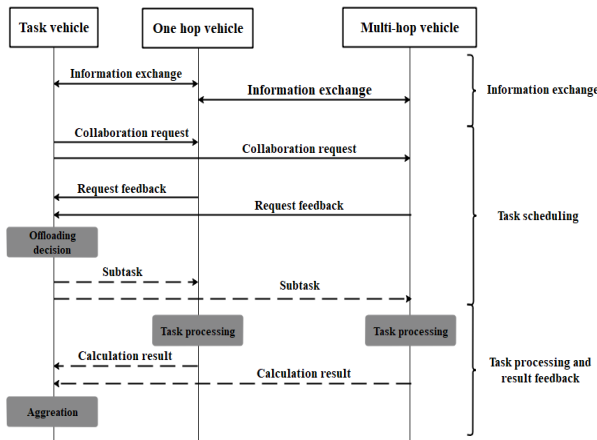


Fig. 4: The flow chart of the proposed offloading scheme

A. Channel Model

1) *Communication Range:* The Nakagami-m distribution can be leveraged in vehicular networks. Through this model, we have the probability density function of received signal power denoted by x as :

$$P(x) = \left(\frac{m}{P_r}\right)^m x^{m-1} e^{-\frac{mx}{P_r}}, \text{ for } x \geq 0, \quad (1)$$

where $\Gamma(\cdot)$, P_d and m represent the Gamma function, the average received power and the fading factor, respectively [41]. Specially, P_d is related to the distance d .

Based on (1), the cumulative distribution function of the communication range of vehicles under the situation that the received power exceeds a threshold P_{th} can be expressed as:

$$F(d) = 1 - \frac{1}{\Gamma(m)} \sum_{i=0}^{m-1} \frac{(m-1)!}{(m-1-i)!} \left(\frac{mP_{th}}{P_r}\right)^{m-1-i} e^{-\frac{mP_{th}}{P_r}}. \quad (2)$$

According to (2), the average communication range of vehicles will be [41]:

$$R = \frac{1}{h\Gamma(m)} \sum_{i=0}^{m-1} \frac{(m-1)!}{(m-1-i)!} \Gamma(m-1-i+\frac{1}{h}) \left(\frac{mP_{th}}{P_t K}\right)^{-\frac{1}{h}}, \quad (3)$$

where h indicates the path-loss exponent. For simplification, it is assumed that the transmission range is same for each vehicle in our work.

2) *Communication Model:* The data transmission rate of per hop communication between two vehicles i and j is given by:

$$r_{ij} = W \cdot \log_2\left(1 + \frac{P_t}{S_{ij}^h P_n}\right), \quad (4)$$

where W represents the channel bandwidth, S_{ij} denotes the inter-vehicle distance, P_t is the transmit power and P_n stands for the background noise power.

B. Mobility Model

In vehicular networks, suffering from the fast mobility of nodes, there is a short connection time between nodes. This will disturb on-going data transmission and can even result in transmission failure [20]. Thus, the link connectivity between nodes plays a paramount role in guaranteeing the offloading success. Here, we first give the definition of link connectivity. After that, the model of link connectivity is presented.

For a pair of nodes i and j , there exists an available link between them if their inter-vehicle distance is not greater than the communication range of nodes. Thus, we use the link connection time between nodes to characterise the link connectivity. Denote $D(t)$ as the inter-vehicle distance between i and j at time t with the initial value d_0 at time 0. Link connection time is defined as the interval during which a direct communication link between i and j is continuously available from the initial time 0 until the moment that the link

between them is disrupted. Let us denote l_{ij} be the available link connection time, then we have:

$$l_{ij} = \{\min t | -R < D(\tau) < -R, D(0) = d_0, 0 < \tau < t\}. \quad (5)$$

Let us define $(x_i, y_i), (x_j, y_j)$ be their coordinates of vehicle i and vehicle j , respectively. Denote v_i, v_j as their speeds, respectively. Define d_{ij} as the their initial inter-vehicle distance: $d_{ij} = x_j - x_i$. According to their movement behaviors, we calculate the link connection time between them for the following several cases:

- two vehicles are moving in the same direction.
If $v_i > v_j$, the link connection time between them can be expressed as:

$$l_{ij} = \frac{R + d_{ij}}{|v_i - v_j|}; \quad (6)$$

otherwise, the link connection time between them will be:

$$l_{ij} = \frac{R - d_{ij}}{|v_i - v_j|}. \quad (7)$$

- two vehicles are moving in opposite directions.
When they are approaching toward each other, the link connection time between them is expressed as:

$$l_{ij} = \frac{\sqrt{R^2 - (y_i - y_j)^2} + d_{ij}}{|v_i + v_j|}; \quad (8)$$

otherwise, the link connection time between them takes the form:

$$l_{ij} = \frac{\sqrt{R^2 - (y_i - y_j)^2} - d_{ij}}{|v_i + v_j|}. \quad (9)$$

C. Task Execution

We consider a client vehicle o with N independent tasks. The set of N tasks is defined as $\mathcal{N} = \{1, \dots, N\}$. The set of client vehicle's service vehicles is defined as $\mathcal{M} = \{1, \dots, M\}$. Each task $i \in \mathcal{N}$ is characterized by a tuple $\{s_i, w_i\}$, where s_i indicates the input data size and w_i specifies the computational resource required for processing this task, i.e., the number of CPU cycles. Client vehicle o can process its tasks locally or by offloading. Let $x_{ki} \in \{0, 1\}$ be the offloading variable of task i , $k \in \mathcal{M}$. If task i is processed locally, $x_{0i} = 1$, otherwise, $x_{0i} = 0$. Meanwhile, if task i is offloaded to service vehicle k , $x_{ki} = 1$, otherwise, $x_{ki} = 0$.

In order to make efficient task offloading decisions, we give the performance analysis of each processing strategy in terms of response delay and computing cost. Besides, service constraints are introduced to reflect the effect of vehicle mobility on task offloading.

1) *Local Execution*: Let f_0 be the local computing capacity of client vehicle o in terms of CPU cycles/s. Then, the time required for client vehicle o to execute task i locally will be:

$$T_{0i} = \frac{w_i}{f_0}. \quad (10)$$

Based on (10), the required time for processing all the tasks that need to be computed locally can be calculated as:

$$T_0 = \sum_{i=1}^N x_{0i} \frac{w_i}{f_0} = \sum_{i=1}^N x_{0i} T_{0i}. \quad (11)$$

2) *Offloading Execution*: We classify service vehicles of client vehicle o into two types: one-hop vehicles and multi-hop vehicles. Let $S_{one}(o)$ and $S_{mul}(o)$ denote the set of one-hop vehicles and multi-hop vehicles, respectively. By using V2V wireless communication, client vehicle o can obtain the information (e.g., position, speed, computation capacity) of its one-hop and multi-hop vehicles. The collected information related to the one-hop and multi-hop vehicles is beneficial for client vehicle o to make an efficient offloading policy.

For any one service vehicle k , the total time to accomplish the assigned task i by client vehicle o denoted by T_{ki} is composed of transmission time T_{ki}^u , execution time T_{ki}^c and feedback time T_{ki}^f :

$$T_{ki} = T_{ki}^u + T_{ki}^c + T_{ki}^f. \quad (12)$$

The time needed to process these tasks from client vehicle o for service vehicle k can be written as:

$$\begin{aligned} T_k &= \sum_{i=1}^N x_{ki} (T_{ki}^u + T_{ki}^c + T_{ki}^f) \\ &= \sum_{i=1}^N x_{ki} T_{ki} \end{aligned} \quad (13)$$

Transmission time for one-hop service vehicles: The time required for uploading task i from client vehicle o to its one-hop service vehicle $k \in S_{one}(o)$ is written as:

$$T_{ki}^u = \frac{s_i}{r_{ok}} \quad (14)$$

where r_{ok} indicates the transmission rate calculated by (4).

Constrained by the mobility of vehicles, the tasks offloaded from client vehicle o to one-hop service vehicles should be finished before the link between the client vehicle and its corresponding service vehicle becomes disconnected. In other words, it is a successful computation offloading only if the whole offloading process is completed before the link between two vehicles becomes unavailable. Let us define l_{0k} as the connection time between client vehicle o and its one-hop vehicle k which can be obtained based on mobility analysis in Section III. One-hop vehicle k is able to process all the assigned tasks if the following equation holds:

$$\sum_{i=1}^N x_{ki} T_{ki} < l_{0k}. \quad (15)$$

Transmission time for multi-hop service vehicles: For any multi-hop vehicle $k \in S_{mul}(o)$, it can receive the information sent by client vehicle o from the intermediate relay nodes, as shown in Fig. 5. Suppose there exist H multi-hop routing path between client vehicle o and its multi-hop vehicle k which form the set $\mathcal{H}$. For any one multi-hop routing path $J \in \mathcal{H}$, it is assumed that there are u one-hop links between client vehicle o and its multi-hop vehicle k . The connectivity of the routing path j can be expressed as:

$$p_j = \min \{l_{o1}, l_{12}, \dots, l_{(u-1)k}\}. \quad (16)$$

Let us take path $o \rightarrow a \rightarrow b \rightarrow k$ for example, where the number associated each link is associated with indicates the corresponding link connection. Then, the connectivity of this routing path is 3.

We select the one among H routing paths with the biggest connectivity for task uploading.

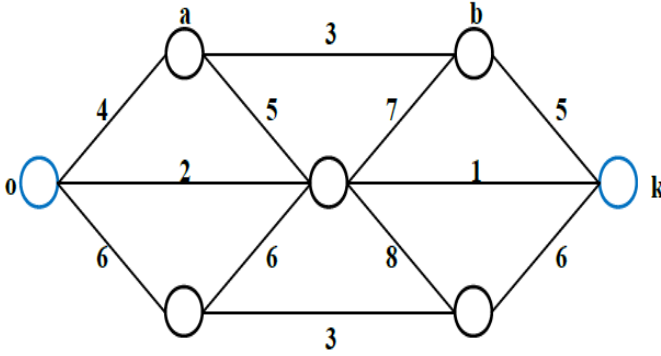


Fig. 5: Example of Multi-hop service vehicles

Task Remote Execution: The execution time needed for any service vehicle k to compute task i is expressed as:

$$T_{ki}^c = \frac{w_i}{f_k}. \quad (17)$$

Calculation Result Feedback: Because the size of the output result is much smaller than the input data, the required time for result feedback is neglected here.

Vehicles in vehicular networks can be either cooperative or selfish. The network will exhibit the better performance when more vehicles participate in resource sharing. In order to motivate vehicles to share resources, the client vehicle should pay to the service vehicles for their resources. To this end, we assume that the unit cost for service vehicle k in task computing is ρ_k . Thus, the total cost needed for service vehicle k to process task i is formulated by:

$$C_{ki} = w_i \rho_k. \quad (18)$$

Based on (18), the total cost paid for all the offloaded tasks to service vehicles is calculated as:

$$C_k = \sum_{i=1}^N x_{ki} w_i \rho_k = \sum_{i=1}^N x_{ki} C_{ki}. \quad (19)$$

IV. MOBILITY-AWARE MULTI-HOP TASK OFFLOADING ALGORITHM IN VEHICULAR EDGE NETWORKS

The quality of experience (QoE) of the client vehicle faces an inherent tradeoff between the execution delay and the cost that it should pay to the service vehicles. Therefore, the utility of the client vehicle is defined as the weighted sum of the total latency and the total cost needed for task processing. Our work aims to minimize the utility of the client vehicle by optimizing the task offloading decisions subject to vehicle mobility.

In the following, we first provide the utility of the client vehicle which is dependent on the execution time and computation cost needed for processing the tasks. Then, an optimization problem is formulated based on the designed utility. Finally, this optimization problem is solved to obtain the desired offloading policy.

A. Utility Design

After receiving the tasks offloaded from the client vehicle, service vehicles can process them in parallel. The time needed for the client vehicle and service vehicles to execute the assigned tasks may be different. In this case, the total execution time is expressed as the longest one for task execution among all the related vehicles, i.e.,

$$t(\mathbf{X}) = \max_{k \in \{0, M\}} T_k = \max_{k \in \{0, M\}} \sum_{i=1}^N x_{ki} T_{ki}, \quad (20)$$

where $\mathbf{X} = \{x_{ki}\} \in \{0, 1\}^{N \times (M+1)}$ indicates the task allocation matrix.

The total computation cost, i.e., the payment provided for service vehicles is written as:

$$c(\mathbf{X}) = \sum_{k \in \mathcal{M}} C_k = \sum_{k \in \mathcal{M}} \sum_{i=1}^N x_{ki} C_{ki}. \quad (21)$$

With the consideration of execution time and computation cost, the offloading utility of the client vehicle can now be expressed as:

$$\begin{aligned} \varphi(\mathbf{X}) &= t(\mathbf{X}) + \lambda_c c(\mathbf{X}) \\ &= \max_{k \in \{0, M\}} \sum_{i=1}^N x_{ki} T_{ki} + \lambda_c \sum_{k \in \mathcal{M}} \sum_{i=1}^N x_{ki} C_{ki}, \end{aligned} \quad (22)$$

where λ_c is a positive weight factor, which indicates the client vehicle's preference on task execution time and computation cost.

B. Problem Formulation

To improve the QoE of the client vehicle, a mobility-aware multi-hop task offloading problem is formulated as the utility minimization problem of the client vehicle given by:

$$\begin{aligned}
\mathcal{P1} : \min_{\mathbf{X}} \varphi(\mathbf{X}) &= t(\mathbf{X}) + \lambda_c c(\mathbf{X}) \\
s.t. \quad C1 : \sum_{k=0}^M x_{ki} &= 1, \forall i \in \mathcal{N} \\
C2 : x_{ki} &\in \{0, 1\}, \forall i \in \mathcal{N}, k \in \{0, M\} \\
C3 : \sum_{i=1}^N x_{0i} T_{0i} &< l_{ok}, \forall k \in S_{one}(o) \\
C4 : \sum_{i=1}^N x_{ki} T_{ki} &< p_{ok}, \forall k \in S_{mul}(o)
\end{aligned} \quad (23)$$

Problem $\mathcal{P1}$ is difficult to solve in general. The constraints in $\mathcal{P1}$ are explained as follows: C1 implies that each task can be computed only at one vehicle; C2 implies that each task can be either processed locally or executed by offloading; C3 implies that the time needed for one-hop service vehicle to process the tasks offloaded from the client vehicle should be less than the connection time between them to guarantee successful task offloading; C4 implies that the multi-hop service vehicle can send the result to the client vehicle before the routing path between them is disconnected, where p_{ok} indicates the connectivity of the routing path for data transmission between the client vehicle and its multi-hop service vehicle, which can be calculated based on (16).

C. Quadratically Constrained Quadratic Programming Transformation and Semidefinite Relation

In order to solve problem $\mathcal{P1}$, we first transform it into a separable quadratically constrained quadratic programming (QCQP) problem. Then, we apply the SDR approach for finding an approximate solution to this problem [42].

1) *QCQP Transformation*: With the introduction of an auxiliary variable t to replace $\max_{k \in \{0, M\}} \sum_{i=1}^N x_{ki} T_{ki}$, we move the delay term in problem $\mathcal{P1}$ to the constraint set. Then, problem $\mathcal{P1}$ is transformed into an equivalent problem $\mathcal{P2}$:

$$\begin{aligned}
\mathcal{P2} : \min_{\mathbf{X}, t} \varphi(\mathbf{X}) &= t + \lambda_c c(\mathbf{X}) \\
s.t. \quad C5 : \sum_{i=1}^N x_{ki} T_{ki} &< t, k \in \{0, M\} \\
C1, C2, C3, C4
\end{aligned} \quad (24)$$

Let us define $\mathbf{w} \triangleq [\mathbf{x}^T, t]^T$, where $\mathbf{x} = \{\mathbf{x}_0^T, \mathbf{x}_1^T, \dots, \mathbf{x}_M^T\}^T$ with $\mathbf{x}_k = \{x_{k1}, x_{k2}, \dots, x_{kN}\}^T, k \in \{0, M\}$. Then, the objective term of problem $\mathcal{P2}$ can be rewritten as:

$$\mathbf{b}^T \mathbf{w}, \quad (25)$$

where $\mathbf{b} = [\lambda_c \mathbf{b}_1^T, 1]^T$, $\mathbf{b}_1 = [\mathbf{d}_1^T, \mathbf{d}_2^T, \dots, \mathbf{d}_M^T]^T$, and $\mathbf{d}_k = [\rho_k w_1, \rho_k w_2, \dots, \rho_k w_N]^T$.

Next, we substitute each constraint in problem $\mathcal{P2}$ by a corresponding matrix form.

For constraint C1, its matrix form is given by:

$$\mathbf{A}_1 \mathbf{w} = \mathbf{1}_{N \times 1}, \quad (26)$$

where

$$\mathbf{A}_1 = \begin{bmatrix} \mathbf{I}_{0,N} & \mathbf{I}_{1,N} & \mathbf{I}_{2,N} & \cdots & \mathbf{I}_{M,N} & \mathbf{0}_{1 \times N} \end{bmatrix}, \quad (27)$$

where $\mathbf{I}$ is a N -dimensional identity matrix.

The integer constraint C2 is replaced by:

$$x_{ki}(x_{ki} - 1) = 0, k \in \{0, M\}, i \in N, \quad (28)$$

and it is further rewritten as the following matrix form:

$$\mathbf{w}^T \text{diag}(\mathbf{e}_p) \mathbf{w} - \mathbf{e}_p^T \mathbf{w} = \mathbf{0}, \quad (29)$$

where $\mathbf{e}_p$ is a $\{N * (M + 1) + 1\} \times 1$ standard unit vector with the p^{th} entry being 1 and $\text{diag}(\mathbf{e}_p)$ is a diagonal matrix, each diagonal element of which corresponds to each one of $\mathbf{e}_p$, respectively.

The constraints C3 and C4 are equivalent to:

$$\mathbf{A}_2 \mathbf{w} < \mathbf{T}_c, \quad (30)$$

where

$$\mathbf{A}_2 = \begin{bmatrix} \mathbf{D}_0^T & \mathbf{0}_{1 \times N} & \mathbf{0}_{1 \times N} & \cdots & \mathbf{0}_{1 \times N} & 0 \\ \mathbf{0}_{1 \times N} & \mathbf{D}_1^T & \mathbf{0}_{1 \times N} & \cdots & \mathbf{0}_{1 \times N} & 0 \\ \mathbf{0}_{1 \times N} & \mathbf{0}_{1 \times N} & \mathbf{D}_2^T & \cdots & \mathbf{0}_{1 \times N} & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{0}_{1 \times N} & \mathbf{0}_{1 \times N} & \mathbf{0}_{1 \times N} & \cdots & \mathbf{D}_M^T & 0 \end{bmatrix}, \quad (31)$$

$$\mathbf{D}_k = [T_{k1}, T_{k2}, \dots, T_{kN}]^T, k \in \{0, M\}, \quad (32)$$

and

$$\mathbf{T}_c = [+ \infty, l_{01}, \dots, l_{0M}]^T. \quad (33)$$

Here l_{0j} indicates the connectivity between two nodes which can be obtained based on mobility analysis in Section III.

The constraint C5 can be expressed as:

$$\mathbf{A}_3 \mathbf{w} < \mathbf{0}_{(M+1) \times 1}, \quad (34)$$

where

$$\mathbf{A}_3 = \begin{bmatrix} \mathbf{D}_0^T & \mathbf{0}_{1 \times N} & \mathbf{0}_{1 \times N} & \cdots & \mathbf{0}_{1 \times N} & -1 \\ \mathbf{0}_{1 \times N} & \mathbf{D}_1^T & \mathbf{0}_{1 \times N} & \cdots & \mathbf{0}_{1 \times N} & -1 \\ \mathbf{0}_{1 \times N} & \mathbf{0}_{1 \times N} & \mathbf{D}_2^T & \cdots & \mathbf{0}_{1 \times N} & -1 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \mathbf{0}_{1 \times N} & \mathbf{0}_{1 \times N} & \mathbf{0}_{1 \times N} & \cdots & \mathbf{D}_M^T & -1 \end{bmatrix}. \quad (35)$$

Based on (25)-(35), problem $\mathcal{P2}$ is further transformed as an equivalent problem $\mathcal{P3}$:

$$\begin{aligned}
\mathcal{P3} : \min_{\mathbf{y}} \quad & \mathbf{b}^T \mathbf{w} \\
s.t. \quad C6 : & \mathbf{A}_1 \mathbf{w} = \mathbf{1}_{N \times 1} \\
C7 : & \mathbf{A}_2 \mathbf{w} < \mathbf{T}_c \\
C8 : & \mathbf{A}_3 \mathbf{w} < \mathbf{0}_{(M+1) \times 1} \\
C9 : & \mathbf{w}^T \text{diag}(\mathbf{e}_p) \mathbf{w} - \mathbf{e}_p^T \mathbf{w} = \mathbf{0}
\end{aligned} \quad (36)$$

By defining $\mathbf{z} \triangleq [\mathbf{w}, 1]^T$, problem $\mathcal{P3}$ is homogenized to a separable QCQP problem:

$$\begin{aligned} \mathcal{P4} : \min_{\mathbf{z}} \quad & \mathbf{z}^T \mathbf{G} \mathbf{z} \\ \text{s.t.} \quad & \text{C10} : \mathbf{z}^T \mathbf{G}_i \mathbf{z} = 1, i = 1, 2, \dots, N \\ & \text{C11} : \mathbf{z}^T \mathbf{G}_j \mathbf{z} < \mathbf{T}_{c,j}, j = 1, 2, \dots, M+1 \\ & \text{C12} : \mathbf{z}^T \mathbf{G}_k \mathbf{z} < 0, k = 1, 2, \dots, M+1 \\ & \text{C13} : \mathbf{z}^T \mathbf{G}_l \mathbf{z} = 0, l = 1, 2, \dots, NM+N \\ & \text{C14} : \mathbf{z} \succeq \mathbf{0} \end{aligned} \quad (37)$$

where

$$\begin{aligned} Q &= N(M+1) + 1, & \mathbf{G} &= \begin{bmatrix} \mathbf{0}_{Q \times Q} & \frac{1}{2} \mathbf{b} \\ \frac{1}{2} \mathbf{b}^T & 0 \end{bmatrix}, \\ \mathbf{G}_i &= \begin{bmatrix} \mathbf{0}_{Q \times Q} & \frac{1}{2} \mathbf{A}_{1,i} \\ \frac{1}{2} \mathbf{A}_{1,i}^T & 0 \end{bmatrix}, & \mathbf{G}_j &= \begin{bmatrix} \mathbf{0}_{Q \times Q} & \frac{1}{2} \mathbf{A}_{2,j} \\ \frac{1}{2} \mathbf{A}_{2,j}^T & 0 \end{bmatrix}, \\ \mathbf{G}_k &= \begin{bmatrix} \mathbf{0}_{Q \times Q} & \frac{1}{2} \mathbf{A}_{3,k} \\ \frac{1}{2} \mathbf{A}_{3,k}^T & 0 \end{bmatrix}, & \mathbf{G}_l &= \begin{bmatrix} \text{diag}(\mathbf{e}_p) & -\frac{1}{2} \mathbf{e}_p^T \\ -\frac{1}{2} \mathbf{e}_p & 0 \end{bmatrix}, \end{aligned}$$

where $\mathbf{T}_{c,j}$ indicates the j th element of $\mathbf{T}_c$, $\mathbf{A}_{1,i}$, $\mathbf{A}_{2,j}$ and $\mathbf{A}_{3,k}$ represent the i th row vector of matrix A_1 , the j th row vector of matrix A_2 and the k th row vector of matrix A_3 , respectively.

2) *Semidefinite Relation*: Optimization problem $\mathcal{P4}$ is difficult to solve. We utilize the SDR approach to relax this problem into a separable semidefinite programming (SDP) problem for finding the solution to problem $\mathcal{P4}$.

Let us define $\mathbf{Z} \triangleq \mathbf{z} \mathbf{z}^T$, then we have $\mathbf{z}^T \mathbf{G} \mathbf{z} = \text{Tr}(\mathbf{G} \mathbf{Z})$, where $\text{rank}(\mathbf{Z}) = 1$. By dropping the constraint $\text{rank}(\mathbf{Z}) = 1$, problem $\mathcal{P4}$ is formulated as a separable SDP problem:

$$\begin{aligned} \mathcal{P5} : \min_{\mathbf{Z}} \quad & \text{Tr}(\mathbf{G} \mathbf{Z}) \\ \text{s.t.} \quad & \text{C15} : \text{Tr}(\mathbf{G}_i \mathbf{Z}) = 1 \\ & \text{C16} : \text{Tr}(\mathbf{G}_j \mathbf{Z}) < \mathbf{T}_{c,j} \\ & \text{C17} : \text{Tr}(\mathbf{G}_k \mathbf{Z}) < 0 \\ & \text{C18} : \text{Tr}(\mathbf{G}_l \mathbf{Z}) = 0 \\ & \text{C19} : \mathbf{Z} \succeq \mathbf{0} \\ & \text{C20} : \mathbf{Z}_{Q+1, Q+1} = 1 \end{aligned} \quad (38)$$

D. Adaptive Adjustment Algorithm

By using standard SDP software, e.g., SeDuMi, we can solve problem $\mathcal{P5}$ in polynomial time and obtain the optimal solution $\mathbf{Z}_{opt}$. Because of the remove of the constraint $\text{rank}(\mathbf{Z}) = 1$ in problem $\mathcal{P5}$, the offloading policy generated from $\mathbf{Z}_{opt}$ will not be equivalent to the one to the original problem $\mathcal{P1}$. In order to deal with this issue, we use an adaptive adjustment procedure [43] to obtain an appropriate offloading policy, including three main steps as follows.

1) *Obtaining an Initial Offloading Solution*: By analyzing $\mathbf{Z}$, it can be found that: $\mathbf{Z}(Q, j) = \mathbf{z}(j)$, $j = 1, 2, \dots, Q$. On the other hand, the first Q elements of $\mathbf{z}$ correspond to the offloading decisions of all the tasks. Thus, we can leverage $\mathbf{Z}(Q, j)$ to recover the offloading decision $\mathbf{z}(j)$.

We regard $\mathbf{Z}(Q, j)$ as the probability of $\mathbf{z}(j)$ being equal to 1. Denote $\mathbf{p} = \{\mathbf{p}_1^T, \mathbf{p}_2^T, \dots, \mathbf{p}_N^T\}^T$ where $\mathbf{p}_k = \{p_{0k}, p_{2k}, \dots, p_{Mk}\}^T$, $k \subseteq \mathcal{N}$ and let $p_{hk} = \mathbf{Z}(Q, h * N + k)$, $h \subseteq \mathcal{M}$. Define $\mathbf{y}_k = [x_{0k}, x_{1k}, \dots, x_{Mk}]^T$, then each element of $\mathbf{y}_k$ is recovered by the following equation:

$$\mathbf{y}_k = \begin{cases} \begin{bmatrix} 1 & 0 & \dots & 0 & 0 \end{bmatrix}^T, & \text{if } \max_{h \in \mathcal{M}} p_{hk} = p_{0k} \\ \begin{bmatrix} 0 & 1 & \dots & 0 & 0 \end{bmatrix}^T, & \text{if } \max_{h \in \mathcal{M}} p_{hk} = p_{1k} \\ \dots & \dots \\ \begin{bmatrix} 0 & 0 & \dots & 1 & 0 \end{bmatrix}^T, & \text{if } \max_{h \in \mathcal{M}} p_{hk} = p_{(M-1)k} \\ \begin{bmatrix} 0 & 0 & \dots & 0 & 1 \end{bmatrix}^T, & \text{if } \max_{h \in \mathcal{M}} p_{hk} = p_{Mk} \end{cases} \quad (39)$$

Let us define $\mathbf{u}_k = [\mathbf{y}_1^T(k), (\mathbf{y}_2)^T(k), \dots, (\mathbf{y}_N)^T(k)]^T$, where $\mathbf{y}_j^T(k)$ indicates the k^{th} element of $\mathbf{y}_j^T$, $j \subseteq \mathcal{N}$, then we can obtain such an initial offloading solution:

$$\mathbf{x}_{ini} = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_{M+1}]^T. \quad (40)$$

2) *Obtaining a Feasible Solution*: There is no guarantee that the generated initial offloading solution $\mathbf{x}_{ini}$ can meet constraint C16. Thus, it is necessary to obtain a feasible offloading solution which satisfies the given constraints once $\mathbf{x}_{ini}$ is not feasible.

Based on $\mathbf{x}_{ini}$, we first select these tasks which need to be executed on service vehicles to form a set S_{off} , i.e., $S_{off} = \{i | \mathbf{y}_i^T(1) = 0\}$, $i \subseteq \mathcal{N}$. Then, check the feasibility of $\mathbf{x}_{ini}$. If $\mathbf{x}_{ini}$ is not feasible, choose randomly $i \in S_{off}$ and modify the offloading decision of task i to be processed locally, i.e., $\mathbf{y}_i^T = [1, 0, \dots, 0]$. Repeat this procedure until $\mathbf{x}_{ini}$ is feasible.

Due to the fact that there is a finite number of available options in S_{off} , the above procedure is guaranteed to converge to a feasible offloading decision.

3) *Obtaining a Local Optimum Solution*: When obtaining a feasible solution, an iterative mechanism is adopted to obtain a local optimum solution to problem $\mathcal{P1}$ by minimizing the client vehicle's utility. The detailed procedure is introduced as follows:

- Make the list of all N tasks from the client vehicle with random order.
- According to this list, examine each task in sequence. For the i^{th} task, it has $(M+1)$ possible offloading decisions. We check each feasible offloading decision of task i without changing the offloading decisions of the other tasks. For any feasible offloading decision of task i , calculate the corresponding utility of this client vehicle. If a lower utility is obtained due to the changed offloading decision, update the solution $\mathbf{x}_{ini}$ and exit this iteration. If i is equal to N , stop this procedure.
- Repeat this procedure until $\mathbf{x}_{ini}$ is converged to a solution.

Because the number of available options in $\mathbf{x}_{ini}$ is limited, the iteration will converge to a solution eventually.

V. NUMERICAL RESULTS

In this section, we implement simulations to demonstrate the performance of our proposed offloading scheme MMTO.

The considered simulation scenario is a unidirectional straight road. Vehicles are uniformly distributed along this road. Let client vehicle be the reference node and located in the original point initially with the speed of 90 km/h. One-hop vehicles and two-hop vehicles are considered for potential service vehicles and their speed follows a uniform distribution between 60 km/h to 120 km/h. For simplicity, high priority is given to the assigned tasks by the client vehicle for each service vehicle with the aim to avoid the queueing process. Besides, we mainly focus on the needed average time for task processing. On the other hand, the distance-based greedy algorithm is used to select the relay node for data transmission. Other simulation parameters are listed Table III unless specified otherwise.

TABLE III: SIMULATION PARAMETERS

Description	Value
Transmission range	100 m
Transmission power	200 mW
Path loss exponent	4
Background noise power	10^{-9} mW
CPU speed of client vehicle	400 MHz
CPU speed of service vehicles	800-1600 MHz
Number of tasks	6
Input size of tasks	0.2 MB-0.3 MB
Computation cost	1/bit
Computation intensity	1000 cycles/bit
Wireless channel bandwidth	5 MHz

We compare the performance of our proposed offloading scheme in terms of delay with the following algorithms:

- **Local Processing Algorithm(APA):** All the tasks of client vehicle are processed locally;
- **Random Policy Algorithm(RPA):** Client vehicle randomly selects available vehicles for task processing.
- **MMTO_O:** The tasks are offloaded to only one hop vehicles in MMTO.
- **RPA_O:** The tasks are offloaded randomly to only one hop vehicles in RPA.

Fig. 6 shows the needed delay for processing all tasks when the number of tasks is varying among three schemes. It is obvious that the increase in the number of tasks enlarges the delay. Due to the limitation of client vehicle in terms of computation capacity, local processing incurs the biggest delay among three schemes. Random policy does better than local processing for the reason that surrounding service vehicles participate in task processing. The cooperation among vehicles enables the delay reduction. However, random policy selects vehicles randomly for task implementation. This method fails to generate the optimal offloading policy, thereby yielding the greater delay than our proposed scheme MMTO. In MMTO, besides one-hop vehicles, multi-hop vehicles are also explored to implement tasks offloaded by the client vehicle, which improves the resource utilization. The optimization problem is designed to reduce the delay, thus the obtained offloading decisions facilitate the fast processing of tasks.

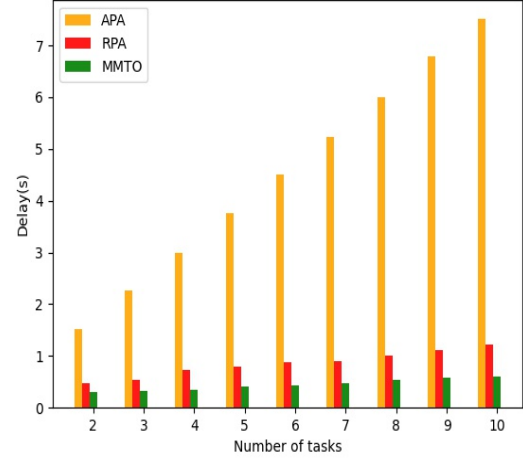


Fig. 6: Delay analysis when the number of tasks is varying

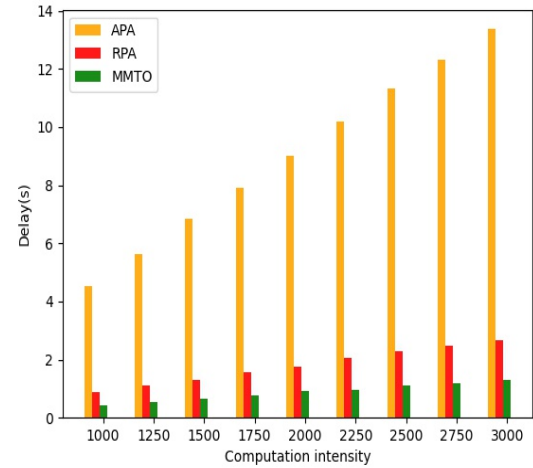


Fig. 7: Delay analysis when the computation intensity is varying

The average delay of three schemes with varying computation intensity, i.e., CPU cycles/bit, is plotted in Fig. 7. From this figure, it can be seen that the delay experienced by client vehicle for processing all tasks is increasing with the increase of computation intensity, showing the same tendency with the result in Fig. 6. The reason behind this result is that when the computation intensity is increased, more computation resources are needed for computing the tasks, thereby contributing to the delay. When changing the computation intensity from 1000 cycles/bits to 3000 cycles/bits, local processing, random policy and MMTO increase the delay up to 13.37 s, 2.65 s and 1.31 s, respectively. It is observed that MMTO achieves the best delay compared to the other schemes. The is because our proposed scheme allows the client vehicle to benefit more from offloading the tasks to surrounding one-hop and multi-hop service vehicles. Thus, the smallest delay variation is brought in case of increasing computation workloads.

The performance of three schemes with respect to the delay against varying number of tasks for both one-hop and multi-hop offloading strategies is shown in Fig. 8. Without asking help from other vehicles, there is no performance difference between two cases for local processing. From this figure, it is found that the incurred delay by random policy and MMTO are smaller than the one experienced by RPA_O and MMTO_O, respectively. This implies that employing two-hop vehicles to compute the offloaded tasks allows leveraging more computation resources for task processing, which is helpful in further shortening the response time. For any case, MMTO performs best since it makes offloading decisions to minimize the delay.

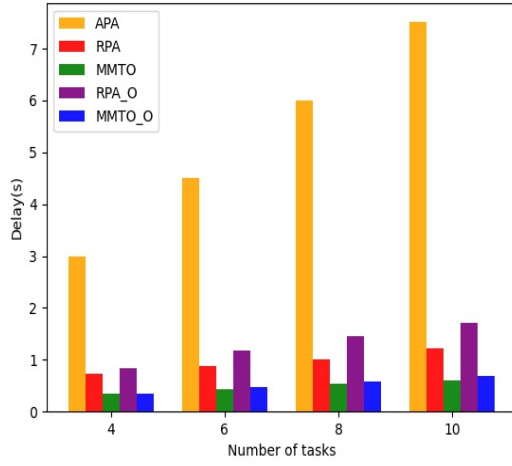


Fig. 8: Delay comparison with varying number of tasks for different offloading strategies

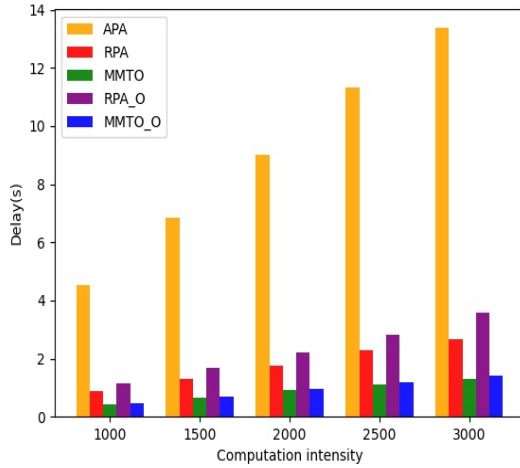


Fig. 9: Delay comparison with varying computation intensity for different offloading strategies

The performance of three schemes in terms of the delay when the computation intensity is varying for both one-hop and multi-hop offloading strategies is illustrated in Fig.

9. Similar with Fig. 8, local processing exhibits the same performance for different cases. On the other hand, more vehicles participate in resource sharing for multi-hop offloading strategy when compared with one-hop offloading strategy, which contributes to reducing the precessing delay.

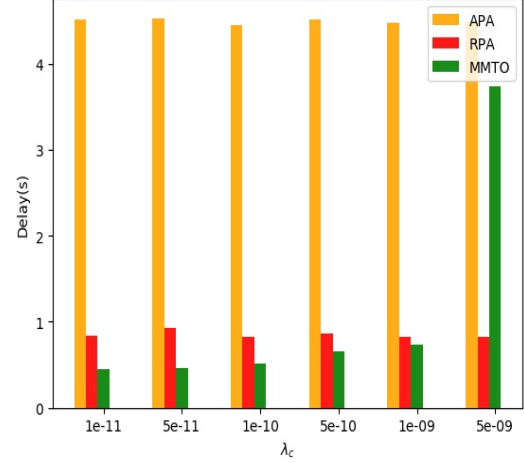


Fig. 10: Delay analysis with varying weight of paid cost

Fig. 10 describes the average time needed for computing all tasks when varying the client vehicle's preference to paid cost, i.e., λ_c , from 1×10^{-11} to 5×10^{-9} . Local processing executes all tasks locally, therefore generating no cost regardless of the variation of λ_c . In random policy, the stochastic delay is caused with varying λ_c based on the fact that each task is offloaded randomly. With increase in λ_c , the cost paid for service vehicles to compute the same size task is increased and the importance of paid cost on the system utility is significantly improved. In this case, more tasks select local processing with the aim to minimize the system utility for MMTO. As a consequence, high delay is incurred in MMTO. When λ_c exceeds 5×10^{-9} , most of the tasks are processed locally, leading to the rapid increase in the delay.

VI. CONCLUSION

In this paper, we focused on the design of vehicle-to-vehicle offloading strategy to improve the resource utilization and enhance user experience. The problem of exploiting mobility task offloading in VEC networks was studied to minimize the utility of client vehicles, which was measured by a weighted sum of execution time and computation cost. Different from existing works, the offloading decisions were made by taking into account both the challenge and the opportunity brought by the mobility of vehicles. Besides one-hop vehicles, two-hop vehicles of client vehicles were employed to facilitate task processing when feasible. A semidefinite relaxation approach was utilized along with an adaptive adjustment algorithm to find a solution for the formulated task offloading problem. The simulation results showed that the proposed vehicle-to-vehicle offloading scheme significantly improves the response delay performance compared to state of the art schemes. blue For

the future work, we will investigate multi-resource on-demand management and task allocation scheme by taking into account the vehicle-to-vehicle and vehicle-to-infrastructure offloading strategies simultaneously for meeting the personalized service demands of different users.

REFERENCES

- [1] K. Yu, L. Lin, M. Alazab, L. Tan, and B. Gu, "Deep learning-based traffic safety solution for a mixture of autonomous and manual vehicles in a 5g-enabled intelligent transportation system," *IEEE Trans. Intell. Transp. Syst.*, 2020, doi:10.1109/TITS.2020.3042504.
- [2] C. Feng, K. Yu, M. Aloqaily, M. Alazab, Z. Lv, and S. Mumtaz, "Attribute-based encryption with parallel outsourced decryption for edge intelligent iov," *IEEE Trans. Veh. Technol.*, vol. 69, no. 11, pp. 13 784–13 795, 2020.
- [3] X. Yu, Y. Chu, F. Jiang, Y. Guo, and D. Gong, "Svms classification based two-side cross domain collaborative filtering by inferring intrinsic user and item features," *Knowl.-Based Syst.*, vol. 141, pp. 80–91, 2018.
- [4] X. Yu, D. Zhan, L. Liu, H. Lv, L. Xu, and J. Du, "A privacy-preserving cross-domain healthcare wearables recommendation algorithm based on domain-dependent and domain-independent feature fusion," *IEEE J. Biomed. Health Informat.*, 2021, doi: 10.1109/JBHI.2021.3069629.
- [5] C. Chen, C. Wang, T. Qiu, M. Atiquzzaman, and D. O. Wu, "Caching in vehicular named data networking: Architecture, schemes and future directions," *IEEE Commun. Surv. Tutor.*, vol. 22, no. 4, pp. 2378–2407, 2020.
- [6] L. Zhao, W. Zhao, A. Hawbani, A. Y. Al-Dubai, G. Min, A. Y. Zomaya, and C. Gong, "Novel online sequential learning-based adaptive routing for edge software-defined vehicular networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 5, pp. 2991–3004, 2020.
- [7] X. Huang, S. Leng, S. Maharjan, and Y. Zhang, "Multi-agent deep reinforcement learning for computation offloading and interference coordination in small cell networks," *IEEE Trans. Veh. Technol.*, vol. 70, no. 9, pp. 9282–9293, 2021.
- [8] W. Sun, N. Xu, L. Wang and H. Zhang, and Y. Zhang, "Dynamic digital twin and federated learning with incentives for air-ground networks," *IEEE Trans. Netw. Sci. Eng.*, 2020, doi:10.1109/TNSE.2020.3048137.
- [9] K. Zhang, J. Cao, and Y. Zhang, "Adaptive digital twin and multi-agent deep reinforcement learning for vehicular edge computing and networks," *IEEE Trans. Ind. Informat.*, 2021, doi:10.1109/TII.2021.3088407.
- [10] L. Zhao, H. Li, N. Lin, M. Lin, C. Fan, and J. Shi, "Intelligent content caching strategy in autonomous driving toward 6g," *IEEE Trans. Intell. Transp. Syst.*, 2021, doi: 10.1109/TITS.2021.3114199.
- [11] K. Zhang, S. Leng, Y. He, S. Maharjan, and Y. Zhang, "Mobile edge computing and networking for green and low-latency internet of things," *IEEE Commun. Mag.*, vol. 56, no. 5, pp. 39–45, 2018.
- [12] L. Liu, C. Chen, Q. Pei, S. Maharjan, and Y. Zhang, "Vehicular edge computing and networking: A survey," *Mobile Netw. Appl.*, 2020, doi:10.1007/s11036-020-01624-1.
- [13] L. Liu, J. Feng, Q. Pei, C. Chen, Y. Ming, B. Shang, and M. Dong, "Blockchain-enabled secure data sharing scheme in mobile-edge computing: An asynchronous advantage actor-critic learning approach," *IEEE Internet of Things J.*, vol. 8, no. 4, pp. 2342–2353, 2021.
- [14] S. Mao, N. Zhang, L. Liu, J. Wu, M. Dong, K. Ota, T. Liu, and D. Wu, "Computation rate maximization for intelligent reflecting surface enhanced wireless powered mobile edge computing networks," *IEEE Trans. Veh. Technol.*, 2021, doi:10.1109/TVT.2021.3105270.
- [15] J. Du, F. R. Yu, G. Lu, J. Wang, J. Jiang, and X. Chu, "Mec-assisted immersive vr video streaming over terahertz wireless networks: A deep reinforcement learning approach," *IEEE Internet of Things J.*, vol. 7, no. 10, pp. 9517–9529, 2020.
- [16] J. Feng, F. R. Yu, Q. Pei, X. Chu, J. Du, and L. Zhu, "Cooperative computation offloading and resource allocation for blockchain-enabled mobile edge computing: A deep reinforcement learning approach," *IEEE Internet of Things J.*, vol. 7, no. 7, pp. 6214–6228, 2019.
- [17] K. Zhang, S. Leng, X. Peng, L. Pan, S. Maharjan, and Y. Zhang, "Artificial intelligence inspired transmission scheduling in cognitive vehicular communications and networks," *IEEE Internet of Things J.*, vol. 6, no. 2, pp. 1987–1997, 2018.
- [18] I. Sorkhoh, D. Ebrahimi, R. Atallah, and C. Assi, "Workload scheduling in vehicular networks with edge cloud capabilities," *IEEE Trans. Veh. Technol.*, 2019, doi:10.1109/TVT.2019.2927634.
- [19] M. M. K. Tareq, O. Semiari, M. A. Salehi, and W. Saad, "Ultra reliable, low latency vehicle-to-infrastructure wireless communications with edge computing," in *IEEE GLOBECOM*, 2018, pp. 1–7.
- [20] K. Zhang, Y. Mao, S. Leng, Y. He, and Y. Zhang, "Mobile-edge computing for vehicular networks: A promising network paradigm with predictive off-loading," *IEEE Veh. Technol. Mag.*, vol. 12, no. 2, pp. 36–44, Jun., 2017.
- [21] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet of Things J.*, vol. 6, no. 3, pp. 4377–4387, 2018.
- [22] Y. Liu, H. Yu, S. Xie, and Y. Zhang, "Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 11, pp. 11 158–11 168, 2019.
- [23] J. Du, F. R. Yu, X. Chu, J. Feng, and G. Lu, "Computation offloading and resource allocation in vehicular networks based on dual-side cost minimization," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1079–1092, 2018.
- [24] Y. Wang, P. Lang, D. Tian, J. Zhou, X. Duan, Y. Cao, and D. Zhao, "A game-based computation offloading method in vehicular multiaccess edge computing networks," *IEEE Internet of Things J.*, vol. 7, no. 6, pp. 4987–4996, 2020.
- [25] K. Zhang, Y. Zhu, S. Leng, Y. He, S. Maharjan, and Y. Zhang, "Deep learning empowered task offloading for mobile edge computing in urban informatics," *IEEE Internet of Things J.*, vol. 6, no. 5, pp. 7635–7647, 2019.
- [26] S. A. Kazmi, T. N. Dang, I. Yaqoob, A. Ndikumana, E. Ahmed, R. Hussain, and C. S. Hong, "Infotainment enabled smart cars: A joint communication, caching, and computation approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 9, pp. 8408–8420, 2019.
- [27] Y. He, N. Zhao, and H. Yin, "Integrated networking, caching, and computing for connected vehicles: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 67, no. 1, pp. 44–55, 2018.
- [28] S. Wang, J. Li, G. Wu, H. Chen, and S. Sun, "Joint optimization of task offloading and resource allocation based on differential privacy in vehicular edge computing," *IEEE Trans. Computat. Social Syst.*, pp. 1–11, 2021.
- [29] F. Hagenauer, C. Sommer, T. Higuchi, O. Altintas, and F. Dressler, "Vehicular micro cloud in action: On gateway selection and gateway handovers," *Ad Hoc Netw.*, vol. 78, pp. 73–83, 2018.
- [30] F. Malandrino, C. Casetti, C.-F. Chiasserini, C. Sommer, and F. Dressler, "The role of parked cars in content downloading for vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 63, no. 9, pp. 4606–4617, 2014.
- [31] G. Qiao, S. Leng, K. Zhang, and Y. He, "Collaborative task offloading in vehicular edge multi-access networks," *IEEE Commun. Mag.*, vol. 56, no. 8, pp. 48–54, 2018.
- [32] Y. Sun, X. Guo, J. Song, S. Zhou, Z. Jiang, X. Liu, and Z. Niu, "Adaptive learning-based task offloading for vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 68, no. 4, pp. 3061–3074, 2019.
- [33] A. U. Rahman, A. W. Malik, V. Sati, A. Chopra, and S. D. Ravana, "Context-aware opportunistic computing in vehicle-to-vehicle networks," *Veh. Commun.*, vol. 24, p. 100236, 2020.
- [34] C.-M. Huang, S.-Y. Lin, and Z.-Y. Wu, "The k-hop-limited v2v2i vanet data offloading using the mobile edge computing (mec) mechanism," *Veh. Commun.*, vol. 26, p. 100268, 2020.
- [35] Y. Qi, Y. Zhou, Y.-F. Liu, L. Liu, and Z. Pan, "Traffic-aware task offloading based on convergence of communication and sensing in vehicular edge computing," *IEEE Internet of Things J.*, 2021, doi: 10.1109/JIOT.2021.3083065.
- [36] N. Tamani, B. Briki, N. Lagraa, and Y. Ghamri-Doudane, "On link stability metric and fuzzy quantification for service selection in mobile vehicular cloud," *IEEE Trans. Intell. Transp. Syst.*, 2019, doi:10.1109/TITS.2019.2911860.
- [37] A. Boukerche and V. Soto, "An efficient mobility-oriented retrieval protocol for computation offloading in vehicular edge multi-access network," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 6, pp. 2675–2688, 2020.
- [38] C. Yang, Y. Liu, X. Chen, W. Zhong, and S. Xie, "Efficient mobility-aware task offloading for vehicular edge computing networks," *IEEE Access*, vol. 7, pp. 26 652–26 664, 2019.
- [39] V. H. Hoang, T. M. Ho, and L. B. Le, "Mobility-aware computation offloading in mec-based vehicular wireless networks," *IEEE Wireless Commun. Lett.*, vol. 24, no. 2, pp. 466–469, 2019.
- [40] C. Zhu, J. Tao, G. Pastor, Y. Xiao, Y. Ji, Q. Zhou, Y. Li, and A. Ylä-Jääski, "Folo: Latency and quality optimized task allocation in vehicular fog computing," *IEEE Internet of Things J.*, 2018, doi:10.1109/JIOT.2018.2875520.

- [41] K. A. Hafeez, L. Zhao, B. Ma, and J. W. Mark, "Performance analysis and enhancement of the DSRC for VANET's safety applications," *IEEE Trans. Veh. Technol.*, vol. 62, no. 7, pp. 3069–3083, Sep., 2013.
- [42] T. Q. Dinh, J. Tang, Q. D. La, and T. Q. Quek, "Offloading in mobile edge computing: Task allocation and computational frequency scaling," *IEEE Trans. Commun.*, vol. 65, no. 8, pp. 3571–3584, 2017.
- [43] M.-H. Chen, M. Dong, and B. Liang, "Resource sharing of a computing access point for multi-user mobile cloud offloading with delay constraints," *IEEE Trans. Mobile Comput.*, vol. 17, no. 12, pp. 2868–2881, 2018.